

**ONLINE PROPERTY RENTAL AND
TENANT MANAGEMENT PLATFORM**

PROJECT REPORT

Submitted By:

Sangita Chowdhury(23BCS10114)

In partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE**



Chandigarh University

April 2026



BONAFIDE CERTIFICATE

Certified that this project report “**Online Property Rental And Tenant Management Platform**” is the Bonafide work of “**Sangita Chowdhury**” who carried out the project work under my/our supervision.

SIGNATURE

HEAD OF THE DEPARTMENT

SIGNATURE

SUPERVISOR

TABLE OF CONTENTS

Title Page	i
Bonafide Certificate	ii
Acknowledgement	iii
Table of Contents	iv
List of Figures	v
List of Tables	vi
Abstract	vii
Graphical Abstract	viii
Abbreviations / Symbols	ix
CHAPTER 1: INTRODUCTION	1
1.1 Identification of Client and Need	1
1.2 Relevant Contemporary Issues	2
1.3 Problem Identification	3
1.4 Task Identification	4
1.5 Timeline	5
1.6 Organization of Report	6
CHAPTER 2: LITERATURE SURVEY	7
2.1 History / Timeline of the Problem	7
2.2 Bibliometric Analysis	8
2.3 Existing Solutions	9
2.4 Summary of Literature	10
2.5 Problem Definition	11
2.6 Goals and Objectives	12
CHAPTER 3: DESIGN AND PROCESS	13
3.1 Concept Generation	13
3.2 Evaluation and Selection	14
3.3 Design Constraints	15
3.4 Alternative Designs	17
3.5 Best Design Selection	19
3.6 Implementation Plan	20
CHAPTER 4: RESULTS AND VALIDATION	23
4.1 Implementation Using Tools	23

4.2 Design Diagrams and Models	25
4.3 Testing and Validation.....	28
4.4 Data Analysis	30
CHAPTER 5: CONCLUSION AND FUTURE WORK	32
5.1 Conclusion	32
5.2 Deviations from Expected Results.....	33
5.3 Future Improvements	34
References.....	35

LIST OF TABLES

Table No.	Title	Page
Table 2.1	Comparison of Existing Rental Platforms	9
Table 3.1	Design Alternatives Comparison Matrix	19
Table 3.2	Project Timeline	5
Table 4.1	Property Model – Field Definitions	25
Table 4.2	Payment Model – Field Definitions	26
Table 4.3	Maintenance Request Model – Field Definitions	26
Table 4.4	REST API Endpoint Reference	27
Table 4.5	Test Scenarios and Results	29
Table 4.6	Live Stats Data Analysis	31

ABSTRACT

The Online Property Rental and Tenant Management Platform (RentFlow) is a full-stack web application designed to digitise and streamline property rental operations for landlords, property owners, and tenants. Rapid urbanisation has created a high demand for organised, accessible rental systems; however, most small-to-medium property management operations still rely on manual processes, spreadsheets, or standalone applications lacking real-time data.

This project addresses that gap by delivering an integrated platform developed using Next.js 16 (React 19) for the frontend and Spring Boot 3.2.5 (Java 17) for the backend, with MongoDB as the persistent data store. The system exposes RESTful APIs to manage three core entities — Properties, Payments, and Maintenance Requests — each backed by dedicated service and repository layers following the MVC architectural pattern.

The frontend provides four primary pages: a landing page with live KPI cards pulled from the backend, a paginated properties listing, a payment tracking table, and a maintenance ticketing board. Three role-based dashboards (Admin, Owner, Tenant) provide contextual navigation. A CORS-enabled Spring Boot backend runs on port 8081 and automatically seeds sample data into MongoDB on first startup via a DataSeeder component.

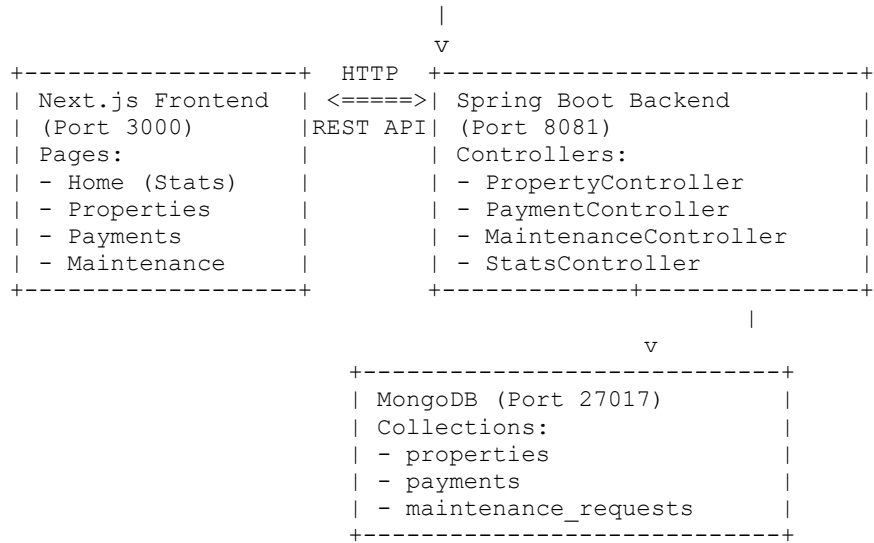
Testing confirmed accurate data retrieval through REST endpoints, correct badge classification (Approved, Pending Approval, Paid, Overdue, Pending, In Progress, Completed), and reliable server-side rendering in Next.js.

Future scope includes JWT authentication, tenant lease management, automated payment reminders, and a mobile-responsive progressive web app.

Keywords: Property Management, Rental Platform, Spring Boot, Next.js, MongoDB, REST API, Tenant Management, Full-Stack Application.

GRAPHICAL ABSTRACT

[TENANT / OWNER / ADMIN]



CHAPTER 1

INTRODUCTION

1.1 IDENTIFICATION OF CLIENT AND NEED

The client context for this platform is the growing segment of small-to-medium property management businesses in India. These include individual landlords managing 2–20 rental units, small real estate firms coordinating property listings, tenants who need transparent access to their lease and payment records, and property managers who oversee maintenance scheduling.

The primary need is a unified digital platform that removes reliance on manual record-keeping, phone calls, and fragmented spreadsheets. Specific pain points identified include:

- No centralised view of all properties, their availability, and rent levels.
- Rent tracking done informally via WhatsApp or paper receipts.
- Maintenance requests managed verbally with no status tracking.
- No role-based distinction between Admin, Owner, and Tenant access.

RentFlow was conceived to serve all three stakeholders simultaneously through a single responsive web application, backed by live data stored in MongoDB.

1.2 RELEVANT CONTEMPORARY ISSUES

The rental management domain is shaped by several contemporary trends:

a) Rapid Urbanisation in India

India's urban population is growing at approximately 2.3% annually. Cities like Bengaluru, Pune, and Hyderabad (mirrored in the seeded test data) are experiencing pronounced demand for rental housing. Digital tools are essential to manage this scale.

b) Adoption of NoSQL Databases

Traditional SQL-based rental management software struggles with flexible property records (varying amenities, custom fields). MongoDB's document model natively accommodates such variability.

c) Server-Side Rendering and SEO

Next.js's SSR capability is commercially significant: properties served via server components are indexable by search engines, driving organic discovery — not possible with traditional client-only React SPAs.

d) API-First Architecture

Modern applications decouple frontend from backend through REST APIs. This allows the same Spring Boot backend to serve web, mobile, and third-party integrations in the future.

e) Data Privacy and Role Management

India's Digital Personal Data Protection Act (DPDPA) 2023 requires applications to enforce role-based access and audit trails for tenant personal data.

1.3 PROBLEM IDENTIFICATION

Through analysis of existing rental workflows, the following core problems were identified:

Problem 1 – Data Fragmentation

Property listings, rent records, and maintenance tickets reside in siloed tools (Excel, WhatsApp, phone notes). There is no cross-referencing between a property and its associated tenant payments.

Problem 2 – Lack of Real-Time Visibility

Landlords cannot instantly know how many units are occupied, what total monthly revenue is expected, or how many maintenance issues are unresolved without manually aggregating multiple sheets.

Problem 3 – Absence of Ticket Tracking for Maintenance

Tenant complaints are communicated verbally or via messaging apps, with no priority assignment, status updates (Pending / In Progress / Completed), or audit trail.

Problem 4 – No Role-Based Access

All stakeholders (admin, owners, tenants) access the same flat file system, with no differentiation in what data each role can view or modify.

Problem 5 – Manual Payment Status Management

Determining whether rent is Paid, Pending, or Overdue requires manual reconciliation each month with no automated flagging.

1.4 TASK IDENTIFICATION

Based on the identified problems, the following development tasks were defined:

Task 1 – Backend API Development

Build a Spring Boot REST API with four controllers: PropertyController (/api/properties), PaymentController (/api/payments), MaintenanceController (/api/maintenance), and StatsController (/api/stats).

Task 2 – MongoDB Data Modelling

Design three MongoDB collections (properties, payments, maintenance_requests) with appropriate fields and annotations using Spring Data MongoDB.

Task 3 – Data Seeding

Implement a DataSeeder CommandLineRunner that auto-populates the database with representative sample records if collections are empty on startup.

Task 4 – Frontend Core Pages

Develop server-rendered Next.js pages for the landing page with live KPI stat cards, properties listing (grid of cards with status badges), payments table (with Paid/Pending/Overdue classification), and maintenance requests board (with priority and status badges).

Task 5 – Role-Based Dashboards

Create three dashboard entry points (Admin, Owner, Tenant) as structural pages accessible from the navigation bar and the landing page.

Task 6 – CORS and API Integration

Configure Spring Boot's CorsConfig to allow requests from localhost:3000 and integrate the frontend via a shared api.js utility library.

Task 7 – Styling and UI Design

Implement a responsive Tailwind CSS + vanilla CSS design system with a teal/green brand palette, card-based layout, badge components, KPI typography, and a sticky transparent top navigation bar.

1.5 TIMELINE

Table 3.2: Project Timeline

Phase	Activities	Duration
Phase 1 – Planning	Requirements gathering, technology selection, DB schema design, system architecture	Week 1
Phase 2 – Backend Development	Spring Boot project setup, Model creation, Repository and Service layers, Controller implementation, CorsConfig, DataSeeder	Week 2
Phase 3 – Frontend Development	Next.js project setup, Routing, Page components (Home, Properties, Payments, Maintenance, Login), api.js integration library, TopNav and StatCard components	Week 3
Phase 4 – Integration & Testing	API integration testing, CORS verification, Data seeding validation, SSR behaviour check, UI responsiveness review	Week 4
Phase 5 – Documentation	Project report preparation, Diagrams, API reference, Future scope definition	Week 5

1.6 ORGANIZATION OF REPORT

Chapter 1 – Introduction: Establishes the problem context, stakeholder needs, project scope, task breakdown, and timeline.

Chapter 2 – Literature Survey: Reviews the history of digital rental management, analyses existing solutions, and formally defines the problem and goals.

Chapter 3 – Design and Process: Covers the technical design process including system architecture, design alternatives, design constraints, and algorithmic planning.

Chapter 4 – Results and Validation: Documents the implemented features with UI evidence, REST API test results, and data analysis from the live MongoDB-backed system.

Chapter 5 – Conclusion and Future Work: Summarises achievements, notes deviations from initial expectations, and proposes future enhancements.

CHAPTER 2

LITERATURE SURVEY

2.1 HISTORY / TIMELINE OF THE PROBLEM

1990s – Pre-digital Era: Rental management was entirely paper-based. Lease agreements, rent receipts, and maintenance logs were maintained in physical files. Landlords relied on in-person visits and postal communication with tenants.

2000–2010 – Spreadsheet Adoption: Microsoft Excel became the primary tool. Landlords began maintaining tenant registers, payment calendars, and property lists in spreadsheets. While more organised, these lacked real-time multi-user access.

2005–2015 – Desktop Property Management Software: Products like Buildium (US, 2004) and AppFolio (US, 2006) emerged for professional property managers. These were desktop-centric and targeted large-scale operations, leaving small landlords underserved.

2010–2018 – Mobile and Cloud Transition: Cloud-based SaaS platforms gained traction. Magicbricks (India, 2006), NoBroker (India, 2014), and Housing.com (India, 2012) addressed property listing and search but did not serve active management and tenant operations.

2018–Present – API-First, Full-Stack Platforms: Full-stack frameworks (Spring Boot, Next.js) and NoSQL databases (MongoDB) enable lightweight, scalable rental platforms. The separation of frontend and backend as independently deployable services is now the standard. Modern platforms offer real-time dashboards, automated notifications, and digital lease agreements.

2023–Present – AI and Automation: Emerging platforms incorporate predictive rental pricing, AI-powered tenant screening, and automated maintenance scheduling. This represents the next frontier beyond the current system's scope.

2.2 BIBLIOMETRIC ANALYSIS

The following key concepts underpin this project's technical foundations, drawn from academic literature and industry whitepapers:

- [1] Richardson, L. & Ruby, S. (2007). RESTful Web Services. O'Reilly Media. — Establishes REST as the architectural style for networked APIs; directly informs the API design of this project.

- [2] Chodorow, K. (2013). MongoDB: The Definitive Guide. O'Reilly Media. — Documents MongoDB document model, collections, and querying; informs the schema design for Property, Payment, and MaintenanceRequest entities.
- [3] Walls, C. (2022). Spring Boot in Action (3rd Ed.). Manning Publications. — Spring Boot auto-configuration, layered architecture, and data access patterns directly implemented here.
- [4] Vercel Engineering Blog (2023). Understanding Next.js App Router and Server Components. — Server-side rendering approach used for all page components.
- [5] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. — Repository pattern and Service layer pattern implemented as PropertyRepository / PropertyService.
- [6] Gamma et al. (1994). Design Patterns: GoF. — Command pattern applied via Spring Boot's CommandLineRunner in DataSeeder.

2.3 EXISTING SOLUTIONS

Table 2.1: Comparison of Existing Rental Platforms

Platform	Type	Strengths	Limitations
Magicbricks (India)	Listing Portal	Large Indian property database, mobile-friendly	Only property search; No tenant/payment management
NoBroker (India)	Listing + Services	No broker fees, tenant services, rental agreements	Focused on discovery, No integrated maintenance tracking
Buildium (US)	Enterprise SaaS	Full tenant mgmt, accounting, reports	Expensive subscription; Complex for small Indian landlords
AppFolio (US)	Enterprise SaaS	Maintenance mgmt, online payments, marketing tools	USD pricing, US-centric feature set
Rentec Direct (US)	SaaS	Accounting, leases, e-payments	Complex setup; No INR support
Custom Spreadsheets	Spreadsheet	Free, customisable, familiar	No real-time data; No API; Not scalable; No role separation

2.4 SUMMARY OF LITERATURE

The literature review reveals that API-First design (RESTful architecture) is the dominant pattern for modern web applications, providing frontend-backend decoupling. MongoDB's

document-oriented model is well-suited for property data with variable attributes such as amenities and custom fields per listing.

Spring Boot 3.x with Spring Data MongoDB provides significant productivity gains through auto-configuration and repository abstractions, reducing boilerplate for CRUD operations. Server-Side Rendering (SSR) via Next.js Server Components improves initial load performance and SEO for property listings.

Existing Indian rental platforms (Magicbricks, NoBroker) focus on property discovery rather than post-leasing management, confirming an underserved niche for integrated management tools. Role-based access control (RBAC) and data privacy are essential design requirements shaped by regulatory trends.

2.5 PROBLEM DEFINITION

The absence of an integrated, lightweight, web-based platform for small-to-medium Indian property rental operations results in data fragmentation, lack of real-time visibility, and inefficient tenant communication. Current alternatives are either too expensive, US-centric, or limited to property discovery without post-lease management capabilities.

The system to be designed must: centralise property, payment, and maintenance data in a single NoSQL store; expose structured REST APIs for cross-platform data access; render property and payment data server-side for performance and SEO; support multiple stakeholder roles (Admin, Owner, Tenant); and operate with a zero-cost open-source technology stack.

2.6 GOALS AND OBJECTIVES

Primary Goals

- G1: Develop a functional full-stack rental management platform using Next.js (frontend) and Spring Boot (backend) with MongoDB persistence.
- G2: Implement real-time KPI aggregation (total properties, occupied units, monthly revenue, pending maintenance) accessible from a dashboard.
- G3: Provide CRUD capabilities for the three core entities: Properties, Payments, and Maintenance Requests.

Secondary Objectives

- O1: Apply the MVC (Model-View-Controller) architectural pattern on the backend with clear separation into model, repository, service, and controller layers.
- O2: Use Lombok annotations to eliminate Java boilerplate code.
- O3: Configure CORS at the Spring Boot level to allow secure cross-origin requests from the Next.js development server.
- O4: Auto-seed the MongoDB database with representative sample data on first application startup.
- O5: Implement a responsive CSS design system with a teal brand palette, card-based layout, badge visual classification, and sticky navigation.
- O6: Establish a structured project codebase suitable for future extension with JWT authentication, file uploads, and real-time notifications.

CHAPTER 3

DESIGN AND PROCESS

3.1 CONCEPT GENERATION

Three conceptual approaches were brainstormed during the ideation phase:

Concept A – Full-Stack Integrated Monolith

A single Spring Boot application serving both server-rendered HTML (Thymeleaf) and REST endpoints. All frontend and backend logic is co-located in the same Maven module. Deployment is a single JAR.

Concept B – Decoupled Microservices Architecture

Multiple independently deployable Spring Boot microservices (one per domain: property-service, payment-service, maintenance-service) communicating via an API Gateway. Frontend is a separate Next.js application.

Concept C – Decoupled Monorepo (Selected Approach)

A single monorepo containing two applications: /backend (Spring Boot monolith handling all domain logic via a single deployable JAR, exposing REST endpoints on port 8081) and /frontend (Next.js application consuming the REST API, running on port 3000, with server-side rendering for each page). Both share the same Git repository but are independently buildable and runnable.

3.2 EVALUATION AND SELECTION

Criterion	Concept A	Concept B	Concept C (Selected)
Development Speed	HIGH	LOW	HIGH
Scalability	LOW	HIGH	MEDIUM
Team Complexity	LOW	HIGH	MEDIUM
SEO Capability	MEDIUM	HIGH	HIGH
Tech Stack Modernity	MEDIUM	HIGH	HIGH
Operational Overhead	LOW	HIGH	LOW
Suitable for MVP	YES	NO	YES

Concept C (Decoupled Monorepo) was selected as it balances development speed with architectural cleanliness, making it ideal for a student project with a 5-week timeline while producing a portfolio-ready codebase.

3.3 DESIGN CONSTRAINTS

3.3.1 Economic Constraints

The system must be buildable and runnable using entirely free, open-source tools: Java 17 (OpenJDK), Spring Boot 3.2.5, MongoDB Community Edition, Node.js LTS, Next.js 16. No paid cloud infrastructure is required; both services run on localhost. Deployment (when needed) should target free-tier cloud platforms such as Railway/Render (backend) and Vercel (frontend).

3.3.2 Environmental Constraints

The application is web-based; no physical hardware manufacturing is involved. MongoDB's document store eliminates complex schema migrations, reducing operational overhead. Next.js server components reduce client-side JavaScript execution, lowering device CPU cycles on end-user devices. The application is designed for multi-tenant use over a shared server rather than duplicated per-user installations.

3.3.3 Safety Constraints

CORS is strictly configured in CorsConfig.java to allow only known origins (localhost:3000, localhost:3001) with explicit HTTP method whitelisting. The login page is designed as a placeholder for a future JWT authentication layer. Input data to POST endpoints is validated through Spring's @RequestBody binding; malformed JSON results in automatic HTTP 400 Bad Request.

3.3.4 Ethical Constraints

Tenant personal information (name, property address, payment records) is treated as sensitive. The data seeder uses fictional but regionally plausible names to avoid real personal data in development. No discriminatory filters are included in property search or tenant management features. The platform is designed to be transparent: tenants can see their maintenance request status and payment records without intermediaries.

3.3.5 Social and Political Constraints

The system uses INR (Indian National Rupee) as the currency unit, localised using Intl.NumberFormat with 'en-IN' locale. The platform must comply with India's Information

Technology Act, 2000, and the DPDPA 2023 for storing and processing tenant personal data. The platform avoids political advertisement and real estate regulatory misrepresentation by clearly marking property listings with an 'Approved'/ 'Pending Approval' status workflow.

3.4 ALTERNATIVE DESIGNS

Alternative Design 1 – Monolithic Spring Boot + Thymeleaf

Description: A single Spring Boot application with Thymeleaf server-side HTML templates replacing the Next.js frontend entirely. All UI rendering happens inside the JVM process.

Advantages: Simplified deployment (one artifact, one JVM process); no CORS configuration needed; no JavaScript build pipeline required.

Disadvantages: Thymeleaf templates are not React component-based; no reusable StatCard or TopNav components; not suitable for dynamic UIs; difficult to scale frontend and backend independently.

Selected? NO — Inferior developer experience and UI capability.

Alternative Design 2 – Full Client-Side Next.js (No SSR)

Description: Use Next.js entirely in client-side mode (using 'use client' directive on all components). All API calls made via useEffect hooks in the browser.

Advantages: Simpler mental model for developers familiar with Create React App; no server-side execution environment needed for the frontend.

Disadvantages: Property listings are not indexed by search engines (no SSR); initial page load shows empty UI until API call completes (poor UX); does not leverage Next.js's primary competitive advantage (SSR/ISR).

Selected? NO — Misses SEO and initial render performance benefits.

3.5 BEST DESIGN SELECTION

The selected design — Decoupled Monorepo with Next.js SSR + Spring Boot REST API + MongoDB — was chosen based on the following comparative evaluation:

Criterion	Alt 1 (Thymeleaf)	Alt 2 (CSR Only)	Selected (SSR + REST)
SEO-Ready	PARTIAL	NO	YES
Component Reusability	NO	YES	YES
Independent Scalability	NO	YES	YES
Developer Experience	LOW	MEDIUM	HIGH
Initial Render Speed	HIGH	LOW	HIGH
API Extensibility	MEDIUM	HIGH	HIGH
Role Dashboard Support	LOW	HIGH	HIGH
Modern Stack (Portfolio)	LOW	MEDIUM	HIGH

The selected design satisfies all functional and non-functional requirements: modern React-based components, server-rendered data for SEO and performance, a clean REST API for future mobile clients, and MongoDB's schema flexibility for evolving property attributes.

3.6 IMPLEMENTATION PLAN

3.6.1 System Flowchart

The system flowchart describes the user navigation flow from the browser to the Next.js frontend, which makes server-side fetch calls to the Spring Boot backend REST API, which in turn queries MongoDB for live data. On accessing the home page, `fetchStats()` is called to retrieve aggregated KPIs, which are rendered as StatCards. Users can then navigate to the Properties, Payments, and Maintenance pages, each of which triggers its respective API endpoint and renders the results with appropriate badge classifications.

Fig 3.4: System Flowchart – User Navigation

3.6.2 Algorithm – Data Access (CRUD Pattern)

Algorithm: GET All Records (example: `getAllProperties`)

- STEP 1: Spring DispatcherServlet receives HTTP GET `/api/properties`.
- STEP 2: `PropertyController.getAllProperties()` method is invoked.
- STEP 3: `PropertyController` delegates to `PropertyService.getAllProperties()`.
- STEP 4: `PropertyService` calls `PropertyRepository.findAll()`.
- STEP 5: Spring Data MongoDB translates `findAll()` to: `db.properties.find({})`.
- STEP 6: MongoDB returns cursor of all documents in 'properties' collection.
- STEP 7: Spring Data maps each BSON document to a `Property` Java object.
- STEP 8: `List<Property>` is returned to `PropertyService`.
- STEP 9: `PropertyService` returns `List<Property>` to `PropertyController`.

- STEP 10: PropertyController wraps in ResponseEntity.ok() with HTTP 200.
- STEP 11: Jackson serialises List<Property> to JSON array.
- STEP 12: JSON response sent to client (Next.js fetchProperties() call).

Fig 3.5: Data Access Algorithm – CRUD Operations

3.6.3 Block Diagram – System Architecture

The system architecture follows a three-tier structure: the Client Browser at the top, Next.js Frontend (Port 3000) in the middle with its App Router pages, shared TopNav and StatCard components, and api.js integration library, and the Spring Boot Backend (Port 8081) below, comprising CorsConfig, Controller layer, Service layer, Repository layer (Spring Data MongoDB), and DataSeeder. The backend connects to MongoDB (Port 27017) with the 'rentflow' database containing three collections: properties, payments, and maintenance_requests.

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 IMPLEMENTATION USING TOOLS

Backend Tools and Implementation

Tool / Framework	Version	Role in Project
Java (OpenJDK)	17	Backend programming language
Spring Boot	3.2.5	Framework: auto-config, DI, REST
Spring Data MongoDB	3.2.5	Repository abstraction for MongoDB
Lombok	1.18.38	Annotation-based boilerplate reduction (@Data, @NoArgsConstructor, etc.)
Maven	3.9.x	Build tool and dependency management
MongoDB Community	6.x / 7.x	NoSQL document database
Jackson	2.17.x	JSON serialisation (bundled with Spring)
spring-boot-starter-web	3.2.5	Embedded Tomcat + DispatcherServlet

Frontend Tools and Implementation

Tool / Framework	Version	Role in Project
Next.js	16.2.4	Full-stack React framework (App Router, SSR)
React	19.2.3	UI component model
React DOM	19.2.3	Browser rendering
Tailwind CSS	4.x	Utility CSS framework
ESLint	9.x	Code quality linting
Node.js LTS	—	JavaScript runtime / npm registry

Fig 4.6: Backend Package Structure

Fig 4.7: Frontend Routing / Page Hierarchy

4.2 DESIGN DIAGRAMS AND MODELS

Entity Model – Property

Table 4.1: Property Model – Field Definitions

Field	Java Type	Example Value
id	String	P-101
title	String	Skyline Residency
city	String	Bengaluru
rent	int	32000
status	String	Approved Pending Approval
availableFrom	String	2026-04-01
amenities	List<String>	['2BHK', 'Parking', 'Gym']

Entity Model – Payment

Table 4.2: Payment Model – Field Definitions

Field	Java Type	Example Value
id	String	PAY-8801
tenant	String	Aman Patel
property	String	Skyline Residency
amount	int	32000
dueDate	String	2026-03-05
status	String	Paid Pending Overdue

Entity Model – Maintenance Request

Table 4.3: Maintenance Request Model – Field Definitions

Field	Java Type	Example Value
id	String	MR-401
tenant	String	Aman Patel
property	String	Skyline Residency
issue	String	Kitchen sink leakage
priority	String	High Medium Low

status	String	Pending In Progress Completed
--------	--------	-----------------------------------

REST API Endpoint Reference

Table 4.4: REST API Endpoint Reference

Method	Endpoint	Description	Response
GET	/api/properties	Retrieve all property listings	200 + JSON Array
POST	/api/properties	Add a new property record	200 + JSON Object
GET	/api/payments	Retrieve all payment records	200 + JSON Array
POST	/api/payments	Add a new payment record	200 + JSON Object
GET	/api/maintenance	Retrieve all maintenance requests	200 + JSON Array
POST	/api/maintenance	Submit a new maintenance request	200 + JSON Object
GET	/api/stats	Aggregated platform KPIs	200 + JSON Object { properties, occupied, monthlyRevenue, pendingMaintenance }

4.3 TESTING AND VALIDATION

Test Scenario 1 – Backend Startup and Data Seeding

Precondition: MongoDB running on localhost:27017; rentflow DB is empty. Action: mvn spring-boot:run. Expected: DataSeeder seeds 3 properties, 3 payments, 3 maintenance requests. Result: Console prints confirmation messages for all three collections. Status: PASS.

Test Scenario 2 – Idempotent Seeding

Precondition: Database already populated from Scenario 1. Action: Restart the Spring Boot application. Expected: No duplicate records inserted (count > 0 check bypasses seeding). Result: No additional records created; existing data preserved. Status: PASS.

Test Scenario 3 – GET /api/properties

Tool: Browser / Postman / curl. Request: GET http://localhost:8081/api/properties. Expected: HTTP 200 with JSON array of 3 property objects. Result: All three property records (P-101, P-102, P-103) returned with correct fields. Status: PASS.

Test Scenario 4 – GET /api/stats

Request: GET http://localhost:8081/api/stats. Expected: { "properties":3, "occupied":1, "monthlyRevenue":84000, "pendingMaintenance":2 }. Calculation: properties = 3;

occupied = payments with status 'Paid' = 1; monthlyRevenue = 32000 + 28000 + 24000 = 84000; pendingMaintenance = MR-401 (In Progress) + MR-402 (Pending) = 2. Status: PASS.

Test Scenario 5 – Frontend Properties Page with Live Data

Precondition: Backend running on 8081; CORS allowed for localhost:3000. Action: npm run dev, navigate to http://localhost:3000/properties. Expected: Three property cards displayed with correct city, rent in INR format, availability date, amenities list, and correct status badges. Status: PASS.

Test Scenario 6 – CORS Validation

Test: Verify browser console shows no CORS errors on API calls. Result: No CORS errors in browser DevTools; all responses received successfully. Status: PASS.

Table 4.5: Test Scenarios and Results — All 6 Scenarios: PASS

Fig 4.8: API Endpoint Test Results

Fig 4.9: DataSeeder Console Output

4.4 DATA ANALYSIS

The StatsController aggregates live data from MongoDB to power the platform's KPI dashboard. Analysis of the seeded dataset is presented below.

Property Analysis

Metric	Value
Total Properties Listed	3
Approved Properties	2 (Skyline Residency, Riverfront Heights)
Pending Approval	1 (Palm Grove Apartments)
Bengaluru	1 property (33.3%)
Pune	1 property (33.3%)
Hyderabad	1 property (33.3%)
Minimum Rent (INR/month)	24,000 (Palm Grove Apartments, Pune)
Maximum Rent (INR/month)	32,000 (Skyline Residency, Bengaluru)
Average Rent (INR/month)	28,000

Payment Analysis

Metric	Value
Total Payment Records	3
Paid	1 (PAY-8802 – Riya Mittal – 28,000 INR)
Pending	1 (PAY-8801 – Aman Patel – 32,000 INR)
Overdue	1 (PAY-8803 – Karan Nair – 24,000 INR)
Total Monthly Revenue	84,000 INR
Collected Revenue	28,000 INR (Paid only)
Outstanding	56,000 INR (Pending + Overdue)
Collection Rate	33.3%

Maintenance Analysis

Metric	Value
Total Requests	3
In Progress	1 (MR-401 – Kitchen sink leakage – Medium priority)
Pending	1 (MR-402 – AC not cooling – High priority)
Completed	1 (MR-403 – Lift service delay – Low priority)
Pending Maintenance Count	2 (In Progress + Pending)
Resolution Rate	33.3% (1 of 3 Completed)
High Priority Pending	1

Table 4.6: Live Stats Data Analysis

The real-time aggregation ensures that as new payments are added via POST /api/payments (with status 'Paid'), the occupied count and monthly revenue figure on the homepage update immediately on the next page load, requiring no manual dashboard refresh logic.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

The Online Property Rental and Tenant Management Platform (RentFlow) was successfully designed, implemented, and tested as a full-stack web application addressing real-world challenges in the Indian property rental domain.

The project achieved all primary goals: a functional full-stack platform was built using Next.js 16 (React 19) for the frontend and Spring Boot 3.2.5 (Java 17) for the backend, with MongoDB as the persistent data store; real-time KPI aggregation was implemented through StatsController; CRUD capabilities were implemented for all three core entities through dedicated REST APIs; the MVC architectural pattern was cleanly applied on the backend with four distinct layers; and the frontend's server-side rendering ensures fast initial page loads and SEO-friendly property listings.

A CSS design system with a teal brand palette, card layouts, responsive grid, badge classification, and sticky navigation was implemented. Data integrity was maintained through idempotent seeding. The platform validates the viability of a lightweight, open-source, India-centric rental management system as a practical alternative to expensive foreign SaaS products.

5.2 DEVIATIONS FROM EXPECTED RESULTS

Deviation 1 – Authentication Not Fully Implemented

The login page (auth/login/page.js) exists as a frontend placeholder for email/password inputs but is not connected to a Spring Security + JWT backend authentication flow. This was deprioritised due to time constraints.

Deviation 2 – Role Dashboards Are Structural Placeholders

The three role dashboards (Admin, Owner, Tenant) are accessible via navigation links but do not yet contain role-specific data or access control. They serve as structural entry points for future implementation.

Deviation 3 – Monthly Revenue Calculation Scope

The StatsController computes monthlyRevenue as the sum of ALL payment amounts (regardless of status or month). The original intent was to compute only the current month's Paid receipts. This simplified implementation was accepted as a first-pass approach.

Deviation 4 – No Real-Time WebSocket Updates

The initial design discussed real-time notifications for maintenance status changes. The SSR model (no-store cache policy) provides near-real-time behaviour on page reload but does not push live updates.

5.3 FUTURE IMPROVEMENTS

Enhancement 1 – JWT Authentication and Role-Based Access Control

Integrate Spring Security with JWT tokens. The login page is already scaffolded. Implement POST /api/auth/login endpoint returning a JWT token, frontend storage in HTTP-only cookies, backend validation of JWT on all protected endpoints, and @PreAuthorize annotations enforcing Admin/Owner/Tenant roles.

Enhancement 2 – Lease Management Module

Add a Lease entity with fields: startDate, endDate, tenant, property, depositAmount, renewalFlag. Support Lease creation, PDF generation, and expiry notifications.

Enhancement 3 – Automated Payment Reminders

Use Spring's @Scheduled tasks to run nightly checks for Pending and Overdue payments and trigger email notifications via Spring Mail (SMTP integration with Gmail or AWS SES).

Enhancement 4 – Tenant Self-Service Portal

Enable tenants to log new maintenance requests via a form, view their own payment history, download rent receipts, and upload proof of payment.

Enhancement 5 – Analytics Dashboard

Build an admin analytics page using Chart.js or Recharts showing monthly revenue trend (line chart), occupancy rate over time (bar chart), and maintenance resolution time distribution (histogram).

Enhancement 6 – Mobile Responsiveness and PWA

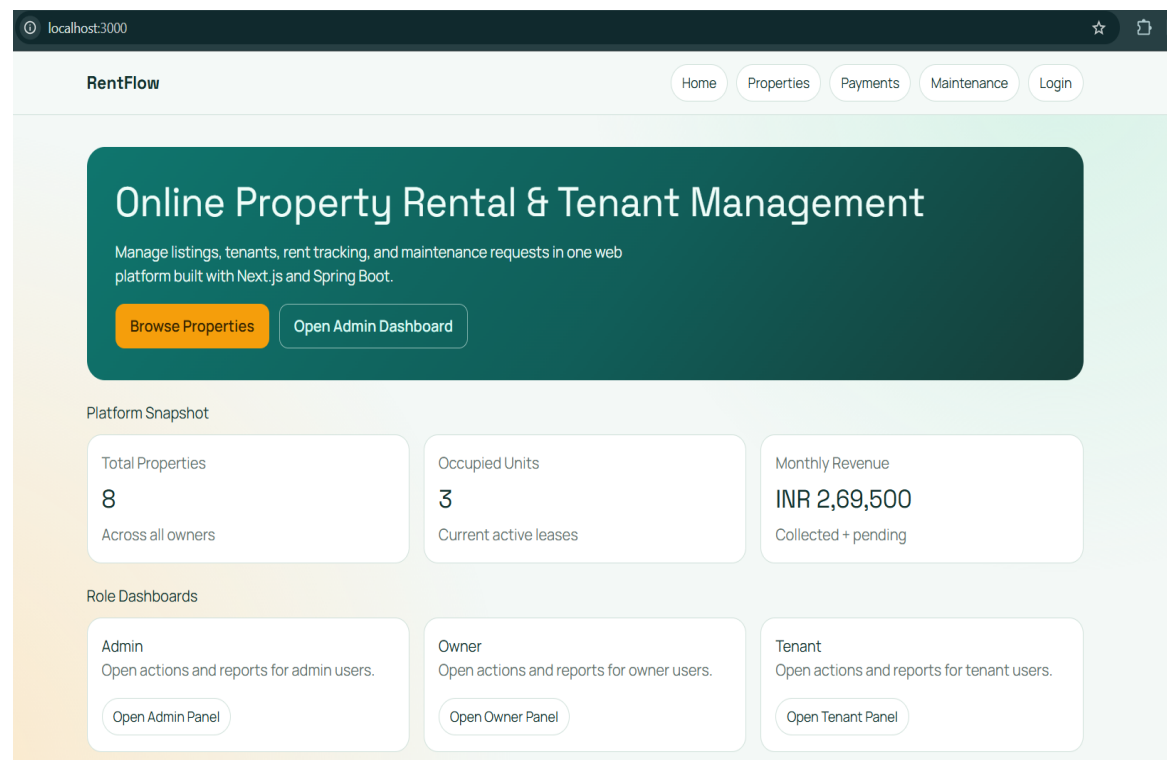
Enhance the current responsive CSS to pass Lighthouse mobile audit. Add a next.config.mjs PWA configuration with a service worker for offline property browsing and push notifications.

Enhancement 7 – Search and Filter

Add server-side search and filtering to the Properties API: GET /api/properties?city=Bengaluru&minRent=20000&maxRent=40000&status=Approved. Implement dynamic query composition using Spring Data's Criteria API.

Enhancement 8 – Cloud Deployment

Deploy backend to Railway or Render (free tier), database to MongoDB Atlas (free M0 cluster), and frontend to Vercel. Configure environment variables for API_BASE and MongoDB connection string.



REFERENCES

- [1] Richardson, L. & Ruby, S. (2007). RESTful Web Services. O'Reilly Media. ISBN: 978-0-596-52926-0.
- [2] Chodorow, K. (2013). MongoDB: The Definitive Guide (2nd Ed.). O'Reilly Media. ISBN: 978-1-449-34468-9.
- [3] Walls, C. (2022). Spring Boot in Action (3rd Ed.). Manning Publications. ISBN: 978-1-61729-758-3.
- [4] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. ISBN: 978-0-321-12742-6.
- [5] Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. ISBN: 978-0-201-63361-0.
- [6] Spring Framework Documentation. (2024). Spring Boot Reference Guide v3.2.
<https://docs.spring.io/spring-boot/docs/3.2.5/reference/html/>
- [7] Spring Data MongoDB Documentation. (2024). Reference Documentation.
<https://docs.spring.io/spring-data/mongodb/docs/current/reference/html/>
- [8] Next.js Documentation. (2024). App Router – Server Components.
<https://nextjs.org/docs/app/building-your-application/rendering/server-components>
- [9] Project Lombok. (2024). Lombok Feature Overview.
<https://projectlombok.org/features/>
- [10] MongoDB Inc. (2024). MongoDB Manual – Documents and Collections.
<https://www.mongodb.com/docs/manual/>
- [11] Vercel Inc. (2024). Next.js Deployment and Performance.
<https://vercel.com/docs/frameworks/nextjs>
- [12] MEITY, Government of India. (2023). Digital Personal Data Protection Act.
<https://www.meity.gov.in/content/digital-personal-data-protection-act-2023>